

200 Interview Questions & Answers: Spring Boot, Microservices & REST APIs

Prepared for: Senior IT Professional / Delivery-Technical Interview Prep **Scope:** 70 Spring Boot | 70 Microservices | 60 REST API questions with detailed answers

Table of Contents

[Section A — Spring Boot \(Q1-Q70\)](#)

[Section B — Microservices \(Q71-Q140\)](#)

[Section C — REST APIs \(Q141-Q200\)](#)

Section A — Spring Boot

Q1. What is Spring Boot and how does it differ from the Spring Framework? Spring Boot is an opinionated extension of the Spring Framework that removes boilerplate configuration. It provides auto-configuration, embedded servers (Tomcat/Jetty/Undertow), starter dependencies, and production-ready features (Actuator) so developers can build stand-alone, production-grade Spring applications with minimal XML/Java config. Spring Framework itself is the underlying IoC/DI container; Spring Boot sits on top of it as a convention-over-configuration layer.

Q2. What are Spring Boot Starters? Starters are curated dependency descriptors (POMs) that bundle a set of compatible libraries for a specific purpose, e.g. `spring-boot-starter-web` pulls in Spring MVC, Jackson, and an embedded Tomcat. They eliminate version-conflict guesswork ("dependency hell") since all versions are managed by the `spring-boot-dependencies` BOM.

Q3. Explain auto-configuration in Spring Boot. Auto-configuration inspects the classpath, existing beans, and properties at startup and automatically registers beans that make sense for the detected environment (e.g., configures a `DataSource` if a JDBC driver is on the classpath). It's implemented via classes annotated `@Configuration` and registered in `META-`

`INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`, guarded by `@Conditional...` annotations so they back off if the user defines their own bean.

Q4. What is `@SpringBootApplication` composed of? It's a meta-annotation combining `@SpringBootConfiguration` (a specialized `@Configuration`), `@EnableAutoConfiguration` (triggers auto-configuration), and `@ComponentScan` (scans the package and sub-packages for components). Placing it on the main class is what bootstraps the entire application context with sensible defaults.

Q5. How does the embedded server work in Spring Boot? Instead of deploying a WAR to an external container, Spring Boot embeds a servlet container (Tomcat by default) inside the fat JAR. On `SpringApplication.run()`, Boot programmatically starts the servlet container in-process, binds it to a port, and deploys the `DispatcherServlet` — making the app a self-contained, directly runnable executable.

Q6. Difference between `@Component`, `@Service`, `@Repository`, and `@Controller`. All four are stereotype annotations that make a class a Spring-managed bean via component scanning. `@Component` is generic. `@Service` marks business-logic classes (semantic clarity, no extra behavior). `@Repository` marks DAO classes and additionally enables Spring's exception-translation, converting JDBC/ORM exceptions into `DataAccessException`. `@Controller` marks Spring MVC controllers returning views; `@RestController` (a combination of `@Controller` + `@ResponseBody`) returns data directly as the response body.

Q7. What is Spring Boot Actuator? Actuator adds production-ready monitoring endpoints (`/actuator/health`, `/metrics`, `/env`, `/beans`, `/loggers`, `/threaddump`, etc.) exposing operational information about the running application. It integrates with Micrometer for metrics export to Prometheus, Datadog, etc., and endpoints can be selectively exposed and secured.

Q8. How do you externalize configuration in Spring Boot? Via `application.properties`/`application.yml`, OS environment variables, command-line args, JNDI, `@ConfigurationProperties`-bound POJOs, and config servers (Spring Cloud Config). Boot merges all sources following a well-defined precedence order so the same artifact can run unmodified across dev/test/prod.

Q9. What are Spring Profiles? Profiles (`@Profile("dev")`, `application-dev.yml`) let you define environment-specific beans and configuration. Activating a profile via `spring.profiles.active` swaps in the matching beans/properties, enabling one codebase to behave differently across environments without code changes.

Q10. Explain the types of Dependency Injection in Spring. Constructor injection (recommended — enables immutability and makes required dependencies explicit, works well with `final` fields and no `@Autowired` needed on a single constructor), setter injection (for optional dependencies), and field injection (`@Autowired` directly on a field — discouraged since it hides dependencies and complicates testing).

Q11. What is `@Autowired` and how does it resolve beans? It instructs Spring's IoC container to inject a matching bean by type. If multiple beans of the same type exist, Spring falls back to matching by field/parameter name, or you disambiguate with `@Qualifier`. `required=false` makes the dependency optional.

Q12. Difference between `@RestController` and `@Controller`. `@Controller` is used for traditional MVC apps where methods return view names resolved to templates. `@RestController` = `@Controller` + `@ResponseBody` on every handler method, so return values are serialized directly (typically to JSON) into the HTTP response body — the standard choice for REST APIs.

Q13. `@RequestMapping` vs `@GetMapping`/`@PostMapping` etc. `@RequestMapping` is the general-purpose mapping annotation where you specify the HTTP method as an attribute. `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@PatchMapping` are composed shortcuts pre-bound to a specific verb, improving readability and reducing errors.

Q14. How does Spring Boot handle centralized exception handling? `@ControllerAdvice` (a global interceptor for controllers) combined with `@ExceptionHandler` methods lets you catch exceptions thrown anywhere in the app and translate them into consistent error responses (often returning a `ProblemDetail`/custom error DTO with proper HTTP status), avoiding repetitive try-catch in every controller.

Q15. What is Spring Data JPA and how does it simplify data access? It's an abstraction over JPA (Hibernate by default) that eliminates boilerplate DAO code. By simply declaring a repository interface (e.g., extending `JpaRepository<Entity, ID>`), Spring generates the implementation at runtime, including CRUD methods and query-derivation from method names (`findByLastNameAndStatus`).

Q16. Explain `@Entity`, `@Table`, `@Id`, `@GeneratedValue`. `@Entity` marks a POJO as a JPA-managed persistent class. `@Table` optionally customizes the mapped table name/schema. `@Id` designates the primary-key field. `@GeneratedValue` specifies the primary-key generation strategy (`IDENTITY`, `SEQUENCE`, `AUTO`, `TABLE`), delegating ID assignment to the database or a persistence provider.

Q17. `JpaRepository` vs `CrudRepository` vs `PagingAndSortingRepository`. `CrudRepository` provides basic CRUD operations. `PagingAndSortingRepository` extends it with pagination/sorting. `JpaRepository` extends both and adds JPA-specific features like batch operations (`saveAll`, `flush`) and returning results as `List` instead of `Iterable`. In practice, most projects extend `JpaRepository` directly.

Q18. What is `@Transactional`? It demarcates a method or class as running within a database transaction, managed declaratively by Spring's AOP proxy. On completion it commits; on a runtime exception (by default) it rolls back. Key attributes: `propagation` (e.g., `REQUIRED`, `REQUIRES_NEW`), `isolation`, `readOnly`, and `rollbackFor`.

Q19. What is Spring Boot DevTools? A development-only dependency enabling automatic restart on classpath changes, LiveReload browser refresh, disabled caching for templates, and helpful defaults for local development — it's automatically excluded from production repackaged JARs.

Q20. How do you configure logging in Spring Boot? Boot uses Commons Logging internally but defaults to Logback. You configure log levels via `application.yml` (`logging.level.com.example=DEBUG`), or supply a `logback-spring.xml` for advanced appenders/patterns. Boot also supports switching to Log4j2 by excluding the default starter and adding `spring-boot-starter-log4j2`.

Q21. Default embedded servlet container and how to switch it. Tomcat is the default. To switch to Jetty or Undertow, exclude `spring-boot-starter-tomcat` from `spring-boot-starter-web` and add `spring-boot-starter-jetty` or `spring-boot-starter-undertow` instead — Boot auto-configures whichever is on the classpath.

Q22. What is `@Value`? Injects a single property value (or SpEL expression) directly into a field/parameter, e.g. `@Value("${server.port}")`. It's suited for simple, individual values; for grouped/structured config, `@ConfigurationProperties` is preferred.

Q23. What is `@ConfigurationProperties`? Binds a hierarchical group of external properties (prefixed, e.g. `app.mail.*`) to a strongly-typed POJO, with IDE auto-completion support (via the annotation processor) and validation support (`@Validated`). It's the recommended way to manage structured configuration versus scattered `@Value` fields.

Q24. `application.yml` vs `application.properties`. Both are supported externalized config formats. YAML supports hierarchical nesting more readably and native lists/maps, while `.properties` is flat key-value. Functionally equivalent — choice is largely a style/readability preference, though YAML is more error-prone with indentation.

Q25. Explain the Spring Bean lifecycle. Instantiation → property population (dependency injection) → `BeanNameAware`/`BeanFactoryAware` callbacks → `BeanPostProcessor.postProcessBeforeInitialization` →

`@PostConstruct` / `InitializingBean.afterPropertiesSet()` / custom init-method →

`BeanPostProcessor.postProcessAfterInitialization` → bean ready for use → on shutdown:

`@PreDestroy` / `DisposableBean.destroy()` / custom destroy-method.

Q26. `@PostConstruct` and `@PreDestroy`. `@PostConstruct` marks a method to run once, right after dependency injection completes, for initialization logic. `@PreDestroy` marks a method to run just before the bean is removed from the container (e.g., on context shutdown), useful for releasing resources like connections or threads.

Q27. Explain the Spring Boot Starter Parent POM. `spring-boot-starter-parent` is a parent POM providing default Maven configuration: dependency management (versions for hundreds of common libraries via a BOM), plugin management (e.g., `spring-boot-maven-plugin` for repackaging into an executable JAR), default Java version, UTF-8 encoding, and resource filtering — drastically reducing POM boilerplate.

Q28. How does Spring Boot achieve "convention over configuration"? By shipping sensible, overridable defaults for nearly everything — default port 8080, embedded Tomcat, JSON via Jackson, an in-memory H2 if on the classpath with no other DataSource configured, default package scanning from the main class location — so a working app requires almost zero explicit configuration, while every default remains overridable via properties or custom beans.

Q29. Explain `@Conditional` and its variants. `@Conditional` lets a bean/configuration be registered only if a condition evaluates true. Boot provides many convenience variants: `@ConditionalOnProperty`, `@ConditionalOnClass` / `@ConditionalOnMissingClass`, `@ConditionalOnBean` / `@ConditionalOnMissingBean`, `@ConditionalOnWebApplication`. These power auto-configuration's "smart defaults that back off" behavior.

Q30. Spring Boot testing annotations: `@SpringBootTest`, `@WebMvcTest`, `@DataJpaTest`. `@SpringBootTest` loads the full application context for integration tests. `@WebMvcTest` loads only the web layer (controllers, `MockMvc`) without the full context, for fast controller-slice tests. `@DataJpaTest` loads only JPA-related components with an in-memory DB and transactional rollback after each test, for repository-slice testing.

Q31. How is Mockito used with Spring Boot tests? Mockito mocks collaborators to isolate the unit under test. `@Mock` creates a mock, `@InjectMocks` injects mocks into the tested object, and in Spring context tests `@MockBean` replaces a real Spring bean with a Mockito mock in the application context — commonly used in `@WebMvcTest` to mock a service layer while testing only the controller.

Q32. Explain Spring Security basics in Spring Boot. Adding `spring-boot-starter-security` auto-configures a default security filter chain (basic auth, CSRF protection, a generated login password). You customize it with a `SecurityFilterChain` bean defining authorization rules (`authorizeHttpRequests`), authentication providers, password encoders (`BCryptPasswordEncoder`), and exception handling for 401/403.

Q33. How to implement JWT authentication in Spring Boot? On login, validate credentials and issue a signed JWT (via `jjwt`/Nimbus) containing claims (subject, roles, expiry). A custom `OncePerRequestFilter` intercepts subsequent requests, extracts the `Authorization: Bearer <token>` header, validates the signature/expiry, and populates the `SecurityContext` with an `Authentication` object so downstream authorization checks work statelessly (no server-side session).

Q34. What is CORS and how do you configure it in Spring Boot? CORS (Cross-Origin Resource Sharing) is a browser security mechanism controlling which origins may call your API from JavaScript. Configure it

globally via a `WebMvcConfigurer` bean overriding `addCorsMappings`, or per-controller/method with `@CrossOrigin`, specifying allowed origins, methods, headers, and credentials.

Q35. How do you customize Actuator health checks? Implement `HealthIndicator` (or `HealthContributor`) and register it as a bean; its `health()` method returns `Health.up().down()` with custom details, which Boot aggregates into the overall `/actuator/health` response — useful for reporting downstream dependency status (DB, message broker, third-party API).

Q36. Explain caching in Spring Boot. `@EnableCaching` activates Spring's caching abstraction. `@Cacheable` caches a method's return value keyed by its arguments (skipping the method body on a cache hit); `@CachePut` always executes and updates the cache; `@CacheEvict` removes entries. The actual store is pluggable (ConcurrentHashMap, Caffeine, Redis, EhCache) via a `CacheManager`.

Q37. What is Spring Batch? A framework for building robust, restartable batch-processing jobs (ETL-style) with chunk-oriented processing (`ItemReader` → `ItemProcessor` → `ItemWriter`), transaction management, job/step metadata persistence for restart-on-failure, and skip/retry policies — commonly used for scheduled bulk data jobs.

Q38. Explain `@Scheduled`. Enabled via `@EnableScheduling`, it lets a method run periodically using `fixedRate`, `fixedDelay`, `initialDelay`, or a cron expression. By default scheduled tasks run on a single-threaded pool, so a custom `TaskScheduler`/`ThreadPoolTaskScheduler` bean is recommended if multiple jobs may overlap.

Q39. What is `@Async`? Enabled via `@EnableAsync`, it makes a method execute on a separate thread from a configured `TaskExecutor`, returning immediately (optionally as `CompletableFuture`/`Future`) instead of blocking the caller — useful for fire-and-forget or parallelizable work like sending emails/notifications.

Q40. How do you handle file uploads in Spring Boot? A controller method accepts `MultipartFile` parameter(s) (Boot auto-configures `MultipartResolver`). You configure `spring.servlet.multipart.max-file-size`/`max-request-size`, then read the file's bytes/stream and persist to disk, a database, or object storage (S3), validating content-type and size before processing.

Q41. Explain global exception handling with `ResponseEntityExceptionHandler`. It's a base class provided by Spring MVC that you extend in a `@ControllerAdvice` to override handling of standard Spring MVC exceptions (e.g., `HttpMessageNotReadableException`, `MethodArgumentNotValidException`) and return consistent `ResponseEntity` error bodies, while still adding your own custom `@ExceptionHandler` methods for domain exceptions.

Q42. `@Bean` vs `@Component`. `@Component` is a class-level annotation picked up by component scanning — you don't control instantiation. `@Bean` is a method-level annotation inside a `@Configuration` class, giving explicit, imperative control over how a bean is constructed — essential for wiring third-party classes you can't annotate directly.

Q43. `ApplicationContext` vs `BeanFactory`. `BeanFactory` is the basic, lazy-initializing IoC container. `ApplicationContext` is a superset adding eager singleton instantiation by default, event publication, internationalization, AOP integration, and easier integration with Spring's various application types — it's what virtually all Spring/Boot apps use.

Q44. How does DataSource auto-configuration work? If a JDBC driver and `spring-boot-starter-data-jpa`/`jdbc` are on the classpath and connection properties (`spring.datasource.*`) are present, Boot auto-

configures a `DataSource` bean (HikariCP by default as the pool), an `EntityManagerFactory`, and a `TransactionManager` — all overridable by defining your own beans.

Q45. How do you connect Spring Boot to multiple databases? Define multiple `DataSource`, `EntityManagerFactory`, and `TransactionManager` beans explicitly (disabling the single auto-configured set), mark one `@Primary`, and separate repository packages per database using `@EnableJpaRepositories(basePackages=..., entityManagerFactoryRef=..., transactionManagerRef=...)` so each repository interface talks to the correct database.

Q46. Flyway/Liquibase with Spring Boot. Both are database schema-migration tools that version-control DDL changes as scripts/changesets. Adding the starter dependency causes Boot to auto-run pending migrations at startup, ensuring the schema is always in a known, reproducible state across environments — critical for CI/CD and team collaboration.

Q47. What is HATEOAS in a Spring context? Spring HATEOAS provides APIs (`EntityModel`, `RepresentationModel`, `WebMvcLinkBuilder`) to build hypermedia-driven responses where resources include navigational `_links` (self, related actions), letting clients discover available operations dynamically rather than hardcoding URLs.

Q48. Does Spring Boot support GraphQL? Yes, via `spring-boot-starter-graphql` (built on `graphql-java`), where you define a schema (`.graphqls`) and implement `@QueryMapping`/`@MutationMapping`/`@SchemaMapping` annotated controller methods, giving GraphQL's flexible, client-driven query model alongside or instead of REST.

Q49. Explain Actuator `/metrics` and Micrometer. Micrometer is a vendor-neutral metrics facade (like SLF4J for metrics) that Actuator uses to record counters, timers, and gauges (JVM memory, HTTP request latency, custom business metrics). It exports to backends like Prometheus, Datadog, or New Relic via pluggable registries, without changing instrumentation code.

Q50. How do you containerize a Spring Boot app with Docker? Write a `Dockerfile` (multi-stage: build with Maven/Gradle, then copy the fat JAR into a slim JRE base image) or use Spring Boot's built-in Buildpacks support (`./mvnw spring-boot:build-image`) which produces an OCI image without writing a Dockerfile, including layered JARs for efficient Docker layer caching.

Q51. Explain Spring Boot's graceful shutdown. Setting `server.shutdown=graceful` makes the embedded server stop accepting new requests but wait (up to `spring.lifecycle.timeout-per-shutdown-phase`) for in-flight requests to complete before shutting down — important in Kubernetes/cloud environments to avoid dropping requests during pod termination/rolling deploys.

Q52. Circuit breaker pattern with Resilience4j. `@CircuitBreaker(name=..., fallbackMethod=...)` wraps a call to an unreliable dependency; after a configured failure-rate threshold within a sliding window, the circuit "opens" and short-circuits calls to the fallback immediately (instead of waiting on timeouts), then periodically allows trial calls ("half-open") to test recovery — preventing cascading failures.

Q53. What is Spring Cloud Config? A centralized externalized-configuration server that serves environment-specific properties from a Git repo (or Vault, filesystem) to all microservices, so config changes can be made and refreshed (`/actuator/refresh` or Spring Cloud Bus) without redeploying services.

Q54. Explain Spring's `Environment` abstraction. `Environment` represents the app's configuration environment: active/default profiles and a unified view over all `PropertySource`s (system properties, env vars, files, etc.), letting code query resolved property values (`environment.getProperty(...)`) irrespective of where they originated.

Q55. Explain Bean scopes. `singleton` (default — one instance per container), `prototype` (new instance every injection/lookup), `request`/`session`/`application` (web-aware scopes tied to an HTTP request/session/ServletContext), and `websocket`. Choosing the wrong scope (e.g., injecting a `prototype` bean into a `singleton` naively) is a common source of subtle bugs.

Q56. `@Primary` and `@Qualifier`. When multiple beans of the same type exist, `@Primary` marks the default candidate for autowiring. `@Qualifier("beanName")` explicitly selects a specific bean by name at the injection point, taking precedence over `@Primary` when both are used together.

Q57. What is Spring AOP? Aspect-Oriented Programming lets you modularize cross-cutting concerns (logging, security, transactions, auditing) as "aspects" applied via pointcuts/advice (`@Before`, `@After`, `@Around`) without scattering that logic through business code. Spring AOP is proxy-based (JDK dynamic proxies or CGLIB), so advice applies only to Spring-managed bean method calls, not internal self-invocations.

Q58. Validation with `@Valid` and `BindingResult`. Annotating a controller parameter with `@Valid` (or `@Validated`) triggers Bean Validation against JSR-380 constraint annotations (`@NotNull`, `@Size`, etc.) on the DTO. Adding a `BindingResult` parameter right after captures validation errors so you can handle them manually instead of Spring throwing `MethodArgumentNotValidException` automatically.

Q59. What is Bean Validation (JSR-380)? A standard (implemented by Hibernate Validator) for declaring constraints directly on model fields (`@NotBlank`, `@Email`, `@Min`, `@Pattern`) that are automatically enforced by frameworks like Spring MVC, keeping validation logic declarative and out of business code.

Q60. `WebClient` vs `RestTemplate`. `RestTemplate` is the classic, synchronous/blocking HTTP client — now in maintenance mode. `WebClient` (from Spring WebFlux) is the modern, non-blocking, reactive client supporting both synchronous (`.block()`) and asynchronous/streaming usage, and is the recommended choice for new development even in non-reactive apps.

Q61. What is Spring WebFlux? Spring's reactive-stack web framework built on Project Reactor (`Mono`/`Flux`) and non-blocking I/O (Netty by default), designed for high-concurrency, low-latency workloads (e.g., streaming, many slow I/O-bound calls) where a small number of threads can handle large numbers of concurrent connections, unlike the thread-per-request model of Spring MVC.

Q62. How is Actuator secured? By combining Spring Security rules for `/actuator/**` endpoints (restricting to authenticated/authorized users), setting `management.endpoints.web.exposure.include` to only the needed endpoints (not `*` in production), and separating the management port (`management.server.port`) from the application port so ops endpoints aren't publicly reachable.

Q63. What is an executable/fat JAR? Spring Boot's Maven/Gradle plugin repackages the app and all its dependencies into a single JAR with a nested-JAR classloader structure (`BOOT-INF/classes`, `BOOT-INF/lib`) and a manifest pointing to `JarLauncher`, so it can be run anywhere with just `java -jar app.jar` without an external application server.

Q64. Externalized configuration precedence order. From highest to lower priority (roughly): command-line args → `SPRING_APPLICATION_JSON` → JNDI/Java System properties → OS environment variables → profile-specific `application-{profile}.yml` → `application.yml` → `@PropertySource` → default properties. Higher-precedence sources override lower ones, which is essential to reason about when debugging "wrong config value" issues.

Q65. What is Spring Boot Admin? A separate community UI application that registers with your Boot apps' Actuator endpoints, giving a dashboard for health, metrics, logs, thread dumps, and environment across all monitored instances — useful for smaller setups without a full observability stack.

Q66. Configuring a `ThreadPoolTaskExecutor`. Define a `@Bean` of type `ThreadPoolTaskExecutor` setting `corePoolSize`, `maxPoolSize`, `queueCapacity`, and `RejectedExecutionHandler`, then reference it by name in `@Async("taskExecutor")` — tuning these correctly (avoiding an unbounded queue that hides backpressure, for example) is critical for predictable async performance.

Q67. `CommandLineRunner` vs `ApplicationRunner`. Both are functional interfaces for running code right after the application context loads and before `SpringApplication.run()` returns. `CommandLineRunner.run(String... args)` receives raw args; `ApplicationRunner.run(ApplicationArguments args)` gets a parsed representation distinguishing option args (`--key=value`) from non-option args.

Q68. What is Testcontainers and how does Spring Boot support it? Testcontainers spins up real, disposable Docker containers (Postgres, Kafka, Redis, etc.) for integration tests instead of mocks/embedded fakes, giving higher-fidelity tests. Spring Boot 3.1+ adds `@ServiceConnection` and `@Testcontainers` integration to auto-wire container connection details into the Spring context with minimal boilerplate.

Q69. What is the N+1 query problem and how do you fix it in Spring Data JPA? It occurs when fetching a parent entity triggers one query, then a separate query per child association (lazy loading in a loop) — N+1 total queries. Fixes: `JOIN FETCH` in a JPQL query, `@EntityGraph` to eagerly fetch specific associations, or enabling Hibernate batch fetching (`hibernate.default_batch_fetch_size`).

Q70. Best practices for structuring a Spring Boot project. Use a layered architecture (controller → service → repository), package by feature/domain rather than by layer for larger apps, keep controllers thin (validation/mapping only), put business logic in services, use DTOs at API boundaries instead of exposing JPA entities, externalize all environment-specific config, and write slice tests alongside full integration tests.

Section B — Microservices

Q71. What are microservices? An architectural style structuring an application as a suite of small, independently deployable services, each owning a single business capability and its own data store, communicating over lightweight protocols (HTTP/REST, messaging). Each service can be developed, deployed, scaled, and even written in a different tech stack independently of the others.

Q72. Monolith vs Microservices. A monolith is a single deployable unit containing all functionality and typically one shared database — simpler to develop and deploy initially but harder to scale/deploy independently as it grows. Microservices trade that simplicity for independent scalability, deployability,

and technology choice, at the cost of operational complexity (distributed systems, network calls, data consistency).

Q73. Advantages and challenges of microservices. Advantages: independent deployability, technology heterogeneity, fault isolation, targeted scaling, smaller codebases easing team autonomy (Conway's Law alignment). Challenges: distributed-systems complexity (network latency/failures), data consistency across services, testing complexity, operational overhead (many deployables to monitor/secure), and the risk of building a "distributed monolith" if boundaries are wrong.

Q74. Domain-Driven Design and bounded contexts. DDD models software around the business domain, using a shared "ubiquitous language" with domain experts. A "bounded context" defines an explicit boundary within which a particular domain model applies consistently — in microservices, bounded contexts are the natural seams along which to split services, avoiding one service leaking another's internal model.

Q75. What is service discovery? Explain Eureka. Since service instances scale up/down and get new IPs dynamically, service discovery lets services find each other by logical name rather than hardcoded addresses. Netflix Eureka is a discovery server where instances register themselves and periodically send heartbeats; clients (often via Spring Cloud LoadBalancer) query Eureka to resolve a logical service name to a live instance.

Q76. What is an API Gateway? A single entry point that sits in front of all microservices, handling cross-cutting concerns like routing, authentication, rate limiting, request/response transformation, and aggregation — so clients don't need to know about internal service topology. Spring Cloud Gateway is the modern reactive replacement for the now-deprecated Netflix Zuul.

Q77. Load balancing in microservices. Server-side load balancing routes through a dedicated load balancer/gateway in front of instances. Client-side load balancing (historically Netflix Ribbon, now Spring Cloud LoadBalancer) has the calling service itself pick an instance from the discovery client's list, distributing load without an extra network hop, commonly combined with round-robin or weighted-response-time strategies.

Q78. Circuit breaker pattern. Prevents a failing/slow downstream service from exhausting caller resources or cascading failure across the system by "opening the circuit" after repeated failures and immediately failing fast (optionally invoking a fallback), then periodically testing whether the dependency has recovered. Resilience4j is the modern standard (Hystrix is deprecated).

Q79. Explain the Saga pattern. Saga manages data consistency across multiple services in a distributed transaction by breaking it into a sequence of local transactions, each publishing an event/message that triggers the next step. If a step fails, compensating transactions undo the preceding steps. It avoids distributed two-phase-commit locks at the cost of eventual (not immediate) consistency.

Q80. What is eventual consistency? A consistency model where, given no new updates, all replicas/services will eventually converge to the same state — but at any given instant, different services may see different (temporarily stale) data. It's the natural consequence of database-per-service microservices and is typically acceptable for many business processes if handled with idempotency and compensations.

Q81. Explain CQRS. Command Query Responsibility Segregation separates the write model (commands, optimized for validation/consistency) from the read model (queries, optimized for fast, often denormalized retrieval) — potentially using entirely different data stores. It pairs naturally with Event Sourcing and helps scale reads and writes independently.

Q82. What is Event Sourcing? Instead of storing only the current state, every state change is persisted as an immutable sequence of events (an append-only event log). Current state is derived by replaying events, giving a full audit trail, the ability to rebuild state/projections, and natural support for temporal queries — at the cost of more complex querying and eventual schema evolution challenges.

Q83. Explain the Strangler Fig pattern. A gradual monolith-to-microservices migration strategy: new functionality is built as microservices while a routing layer (facade/gateway) incrementally redirects traffic from old monolith code paths to the new services, until the monolith is fully "strangled" and can be retired — much lower-risk than a big-bang rewrite.

Q84. What is a service mesh? Infrastructure (e.g., Istio, Linkerd) that handles service-to-service communication concerns — mTLS, retries, load balancing, observability, traffic shaping (canary/blue-green) — via lightweight sidecar proxies injected next to each service instance, removing that logic from application code entirely.

Q85. Distributed tracing: Sleuth, Zipkin, Jaeger. Since a single business request may traverse many services, distributed tracing propagates a correlation/trace ID (and span IDs per hop) through every call. Spring Cloud Sleuth (now largely superseded by Micrometer Tracing) instruments this automatically; Zipkin/Jaeger collect and visualize the traces as a timeline, making cross-service latency and failures debuggable.

Q86. Centralized logging with the ELK stack. Since logs are scattered across many service instances/pods, centralized logging ships all logs (via Filebeat/Fluentd/Logstash) to Elasticsearch for indexing and Kibana for search/visualization, correlated by trace IDs so you can reconstruct the full path of a request across services from one place.

Q87. Database-per-service pattern. Each microservice owns its private datastore, accessible only through that service's API — no other service may query it directly. This enforces true encapsulation and independent evolvability/scaling of each service's schema, but requires patterns like Saga/API composition/CQRS to handle queries and transactions that span multiple services.

Q88. API Composition pattern. When a query needs data owned by multiple services (which each have their own DB), an API Composer (often at the gateway or a dedicated aggregator service) invokes each owning service and combines the results in memory — simple to implement but can be inefficient for large in-memory joins across many services.

Q89. Synchronous vs asynchronous communication. Synchronous (typically REST/gRPC request-response) is simple and easy to reason about, but couples the caller's availability/latency to the callee's. Asynchronous (message/event-based via Kafka/RabbitMQ) decouples services in time and space, improves resilience, but introduces eventual consistency and more complex debugging/monitoring.

Q90. Role of message brokers (Kafka, RabbitMQ). Brokers decouple producers from consumers, buffering messages so services can process at their own pace, enable event-driven architectures (publish once, many subscribers), and provide durability/replay. Kafka is a distributed log optimized for high-

throughput streaming and replay; RabbitMQ is a traditional message broker optimized for flexible routing and lower-latency task queuing.

Q91. Idempotency in microservices. An idempotent operation produces the same result no matter how many times it's applied — essential because network retries (from timeouts, at-least-once message delivery) can cause the same request/message to be processed more than once. Achieved via idempotency keys, unique constraint checks, or upserts, so retries are safe.

Q92. What is the Outbox pattern? To atomically update the local database and publish an event (avoiding dual-write inconsistency), a service writes the event to an "outbox" table in the same local transaction as the business data change; a separate relay process (or CDC tool like Debezium) then reliably publishes those outbox rows to the message broker.

Q93. Bulkhead pattern. Named after ship compartments — isolates resources (thread pools, connection pools) per downstream dependency so that if one dependency becomes slow/saturated, it can't exhaust resources needed by calls to other, healthy dependencies, containing the blast radius of a failure.

Q94. Retry pattern and backoff strategies. Automatically re-attempts a failed (typically transient) operation instead of failing immediately. Naive immediate retries can worsen an overloaded dependency, so exponential backoff (increasing delay between attempts, often with jitter to avoid thundering-herd synchronization) is used, combined with a max-attempts cap and ideally circuit breaking.

Q95. Rate limiting in microservices. Restricts the number of requests a client/service can make in a time window, protecting services from overload/abuse. Common algorithms: token bucket, leaky bucket, fixed/sliding window counters. Typically enforced at the API Gateway (Spring Cloud Gateway's `RequestRateLimiter`, often backed by Redis) rather than in each service.

Q96. What is the Sidecar pattern? Deploys a helper component (a "sidecar" container/process) alongside the main service instance to handle cross-cutting concerns (logging, config, proxying, service-mesh networking) without modifying the primary application — the sidecar shares the lifecycle and network namespace of the main container, commonly seen in Kubernetes pods and service meshes.

Q97. Kubernetes basics for microservices. Kubernetes orchestrates containers: a Pod is the smallest deployable unit (one or more co-located containers), a Deployment manages desired replica count and rolling updates of Pods, a Service provides a stable virtual IP/DNS name and load balancing across Pod replicas, and a ConfigMap/Secret externalizes configuration/credentials — together enabling self-healing, scalable microservice deployments.

Q98. What is a Config Server? A dedicated service (e.g., Spring Cloud Config Server) that centrally stores and serves externalized configuration for all microservices from a versioned source (typically a Git repo), so config changes are auditable, environment-specific, and can be applied without rebuilding/redeploying the service binaries.

Q99. Versioning strategies for microservice APIs. URI versioning (`/v1/orders`), header/media-type versioning (`Accept: application/vnd.company.v2+json`), or query-param versioning. The key discipline regardless of mechanism is maintaining backward compatibility where possible (additive changes) and giving consumers a clear deprecation timeline for breaking changes.

Q100. Consumer-Driven Contract testing (Pact). Consumers define a "contract" (expected request/response) against a provider; the contract is verified against the actual provider in CI. This catches breaking API changes early without needing full end-to-end integration environments, and is especially valuable in microservices where many teams own many independently-deployed services.

Q101. Backend for Frontend (BFF) pattern. Instead of one generic API Gateway serving all client types, each distinct client (web, mobile, third-party) gets its own tailored backend that aggregates/shapes data specifically for that client's needs — avoiding an overly generic, bloated gateway API and letting each client evolve its BFF independently.

Q102. Service-to-service authentication (mTLS, OAuth2 client credentials). mTLS has both client and server present certificates, mutually verifying identity at the transport layer — often handled transparently by a service mesh. OAuth2 client-credentials grant lets a service authenticate itself (not on behalf of a user) to obtain an access token for calling another service's API, verified typically via JWT signature/introspection.

Q103. Distributed transactions: 2PC vs Saga. Two-Phase Commit gives strong (ACID) consistency across resources but requires a coordinator holding locks across all participants until every one votes to commit — poorly suited to microservices' independent deployability/availability goals and doesn't scale well across service/network boundaries. Saga trades strong consistency for availability/scalability using local transactions plus compensations, aligning better with microservices' autonomy.

Q104. What is chaos engineering? The practice of deliberately injecting failures (killing instances, adding latency, saturating resources) into a system, often in production, to proactively verify resilience mechanisms (retries, circuit breakers, failover) actually work as intended before a real outage does — pioneered by Netflix's Chaos Monkey.

Q105. Blue-green vs canary deployment. Blue-green runs two full identical environments; traffic is switched entirely from the old (blue) to the new (green) version at once, enabling instant rollback. Canary gradually shifts a small percentage of traffic to the new version, monitoring for errors before progressively increasing it — lower blast radius but slower and requires good metrics/automation.

Q106. Twelve-Factor App methodology. A set of best practices for building cloud-native, scalable SaaS apps: one codebase per app, explicit dependency declaration, config stored in the environment (not code), treating backing services as attachable resources, strict separation of build/release/run stages, stateless processes, port-binding, concurrency via the process model, fast startup/graceful shutdown ("disposability"), dev/prod parity, logs as event streams, and admin tasks as one-off processes.

Q107. Health check patterns. Liveness checks answer "is the process alive and should it be restarted if not?" Readiness checks answer "is the instance ready to receive traffic right now?" (e.g., DB connection established). Kubernetes uses both separately — a service can be alive but not ready, which should remove it from load-balancer rotation without killing the pod.

Q108. Saga orchestration vs choreography. Orchestration uses a central coordinator that explicitly tells each service what local transaction to execute and in what order — easier to understand/monitor centrally but introduces a coordinating component. Choreography has each service publish events that other services react to autonomously, with no central coordinator — more decoupled but harder to trace overall workflow state.

Q109. Data consistency patterns across services. Options include Saga (eventual consistency via compensations), CQRS with eventually-consistent read models, the Outbox pattern for reliable event publication, and API Composition for cross-service reads. The choice depends on whether the business truly needs strong consistency (rare across service boundaries) or can tolerate a brief eventual-consistency window.

Q110. Kafka's role in event-driven microservices. Kafka acts as a durable, partitioned, replicated event log/backbone: services publish domain events (OrderCreated, PaymentProcessed) to topics, and any number of interested consumers subscribe independently, enabling loose coupling, replayability (new consumers can process historical events), and high-throughput asynchronous integration between services.

Q111. Schema evolution in Kafka (Avro/Schema Registry). Since producers and consumers evolve independently, a Schema Registry stores versioned Avro/Protobuf schemas and enforces compatibility rules (backward/forward/full) when a producer registers a new schema version, preventing consumers from breaking due to unexpected field changes in the event payload.

Q112. CAP theorem and microservices. CAP states a distributed system can guarantee at most two of Consistency, Availability, and Partition tolerance during a network partition. Since partitions are unavoidable in distributed systems, the real trade-off is C vs A during a partition — most microservice architectures favor Availability with eventual Consistency (AP) over strict consistency, given business tolerance for staleness.

Q113. Distributed caching (Redis) in microservices. Redis is often used as a shared, low-latency cache layer (session data, frequently-read reference data, rate-limiter counters) accessible across service instances, reducing database load and improving response times, while requiring an explicit cache-invalidation strategy (TTL, write-through, or event-driven eviction) to avoid serving stale data.

Q114. What is a Dead Letter Queue (DLQ)? A separate queue/topic where messages that repeatedly fail processing (after retry exhaustion) are routed instead of being lost or blocking the main queue indefinitely — allowing operators to inspect, fix, and reprocess failed messages without impacting the throughput of successfully-processed ones.

Q115. Microservices testing pyramid. Bottom (largest volume): fast, isolated unit tests. Middle: integration tests (against a real DB/broker via Testcontainers) and consumer-driven contract tests verifying inter-service API compatibility without full deployment. Top (smallest volume, slowest, most brittle): end-to-end tests across multiple deployed services — kept minimal since they're expensive to maintain and slow to run.

Q116. REST vs gRPC for inter-service communication. REST/JSON over HTTP/1.1 is simple, human-readable, and broadly supported — a good default, especially for external/public APIs. gRPC uses HTTP/2 and Protocol Buffers for compact binary payloads, supports streaming (unary, server/client/bidirectional), and offers strongly-typed contracts with generated client/server code — often preferred for high-throughput, low-latency internal service-to-service calls.

Q117. API versioning best practices. Prefer additive, backward-compatible changes (new optional fields/endpoints) over breaking changes wherever possible; when a breaking change is unavoidable, version explicitly, support the old version in parallel for a defined deprecation window, communicate via

changelogs/headers (`[Deprecation]`, `[Sunset]`), and use consumer-driven contract tests to catch accidental breakage.

Q118. Deciding microservice boundaries/granularity. Boundaries should align with bounded contexts/business capabilities (DDD), not technical layers — a service should own a cohesive piece of business functionality and its data. Signals for splitting include independent scaling needs, differing release cadences, and team ownership boundaries (Conway's Law); over-splitting leads to a "distributed monolith," under-splitting delays realizing microservice benefits.

Q119. What is a "Distributed Monolith" anti-pattern? Occurs when services are physically separated (deployed independently) but remain tightly coupled logically — e.g., synchronous call chains requiring all services to be up simultaneously, a shared database, or lock-step deployments — yielding all the operational complexity of microservices with none of the independence benefits.

Q120. Shared library anti-pattern. Sharing a common library containing business domain models/logic across services seems to reduce duplication, but it recreates tight coupling: a change to the shared library forces coordinated redeployment of every consuming service, undermining independent deployability. Shared technical/infrastructure libraries (logging, tracing) are fine; shared domain models generally are not.

Q121. Zero-downtime deployment strategies. Rolling updates (replace instances gradually while others keep serving traffic), blue-green (instant cutover between two environments), and canary (gradual traffic shift with monitoring) all avoid a maintenance window. They require the service to support running two versions simultaneously (backward/forward-compatible API and DB schema changes) during the transition.

Q122. Role of Docker Compose in local microservices development. Docker Compose lets developers define and spin up an entire multi-service stack (app services, databases, message brokers, mock dependencies) with one command via a declarative YAML file, closely mirroring production topology for local development/integration testing without needing a full Kubernetes cluster.

Q123. Kubernetes Deployment, Service, Pod. A Pod wraps one or more tightly-coupled containers sharing network/storage. A Deployment declares the desired state (replica count, container image, update strategy) and reconciles actual Pods to match it, handling rolling updates/rollbacks. A Service provides a stable network identity (ClusterIP/NodePort/LoadBalancer) and load-balances traffic across the matching Pods regardless of their ever-changing individual IPs.

Q124. Horizontal Pod Autoscaling. The Horizontal Pod Autoscaler automatically adjusts the number of Pod replicas in a Deployment based on observed metrics (CPU/memory utilization, or custom metrics like request rate/queue depth), scaling out under load and back in during quiet periods, without manual intervention.

Q125. Three pillars of observability. Metrics (aggregated numeric time-series like request rate/latency/error rate — good for dashboards/alerting), Logs (discrete, detailed event records — good for deep debugging of a specific incident), and Traces (the end-to-end path of a single request across services — good for pinpointing where in a distributed call chain time was spent or a failure occurred). Together they give complementary visibility a single pillar can't.

Q126. Strangler pattern vs Big Bang migration. Big Bang rewrites and cuts over the entire monolith to microservices at once — high risk, all-or-nothing, long time before delivering value. Strangler Fig migrates incrementally, routing traffic feature-by-feature to new services while the monolith keeps running the rest, delivering value continuously and de-risking the migration significantly.

Q127. Feature flags/toggles in microservices. Feature flags let you enable/disable functionality at runtime (often per-user-segment or percentage rollout) without a redeploy, decoupling deployment from release. This supports safer canary releases, quick rollback of a problematic feature, and trunk-based development where incomplete features can be merged but hidden behind a flag.

Q128. API throttling and quota management. Throttling limits request rate over short windows to protect infrastructure from spikes/abuse (often per API key/client), while quotas cap total usage over a longer period (e.g., 10,000 calls/day) typically tied to a subscription tier — both usually enforced at the API Gateway with clear `429 Too Many Requests` responses and `Retry-After`/rate-limit headers.

Q129. Secrets management (Vault) in microservices. HashiCorp Vault (or cloud-native equivalents like AWS Secrets Manager) centrally stores and tightly access-controls sensitive credentials (DB passwords, API keys, certificates), issuing short-lived, dynamically-generated secrets to services at runtime instead of embedding static secrets in config files or environment variables, reducing blast radius if a secret leaks.

Q130. Role of Spring Cloud in building microservices. Spring Cloud provides a cohesive set of libraries solving common microservices concerns on top of Spring Boot: service discovery (Netflix Eureka/Consul), client-side load balancing, API gateway (Spring Cloud Gateway), centralized config (Config Server), resilience (via Resilience4j integration), and distributed tracing — letting teams assemble a microservices platform without building this infrastructure from scratch.

Q131. Distributed locking (Redisson, ZooKeeper). When multiple service instances must ensure only one executes a critical section at a time (e.g., a scheduled job that must run exactly once across replicas), a distributed lock coordinated through a shared system (Redis via Redisson, or ZooKeeper/etcd) provides mutual exclusion across process/machine boundaries, unlike an in-JVM lock which only works within one process.

Q132. Transactional Outbox vs Change Data Capture (Debezium). Both solve reliable event publication after a local DB write. The Outbox pattern requires application code to explicitly write to an outbox table and a relay to publish it. Debezium/CDC instead reads the database's transaction log directly (no application code changes needed) and streams row-level changes to Kafka, making it less invasive but tied to a specific database's log format.

Q133. Multi-tenancy patterns in microservices. Separate database per tenant (strongest isolation, highest operational cost), shared database with a tenant-ID column/schema-per-tenant (balances isolation and cost), or a fully shared schema with row-level security. The right choice depends on tenant data-isolation/compliance requirements, expected tenant count, and noisy-neighbor tolerance.

Q134. API Gateway's cross-cutting responsibilities. Centralizes concerns that would otherwise be duplicated in every service: authentication/authorization, TLS termination, request routing, rate limiting, request/response transformation, response caching, and centralized logging/metrics for all inbound traffic — simplifying individual services which can then focus purely on business logic.

Q135. Graceful degradation and fallback methods. When a non-critical dependency fails, the system should degrade gracefully rather than fail entirely — e.g., a circuit breaker's fallback method returns cached/default data or a simplified response instead of an error, preserving core functionality for the user even when a secondary feature (like personalized recommendations) is temporarily unavailable.

Q136. Synchronous call chaining vs event-driven choreography for workflows. Chaining synchronous REST calls (Service A calls B calls C) is simple to trace but couples all services' availability/latency together and risks cascading timeouts. Event-driven choreography publishes events that trigger subsequent steps asynchronously, decoupling availability and improving resilience, at the cost of harder-to-trace overall workflow state and eventual consistency.

Q137. Microservice CI/CD pipeline considerations. Each service should have its own independent pipeline (build, unit/integration/contract tests, containerize, deploy) so one team's release doesn't block another's. Pipelines typically include automated contract tests against dependent services, security/dependency scanning, progressive deployment strategies (canary), and automated rollback triggers on health-check/error-rate degradation.

Q138. Service Registry vs Config Server. A Service Registry (Eureka/Consul) tracks *where* live service instances currently are (dynamic network locations) for discovery/load-balancing purposes. A Config Server tracks *what* configuration values each service/environment should use. They solve different problems and are typically both present in a Spring Cloud-based microservices platform.

Q139. Common pitfalls decomposing a monolith into microservices. Splitting along technical layers instead of business capabilities, sharing a single database across "independent" services, ignoring the network's unreliability (treating remote calls like local calls), skipping automated testing/observability investment, and underestimating the organizational/DevOps maturity required to operate many independently-deployed services well.

Q140. Best practices for designing resilient microservices. Design for failure by default (timeouts, retries with backoff, circuit breakers, bulkheads), keep services stateless where possible, ensure idempotent operations, invest early in observability (metrics/logs/traces), automate deployment/rollback, enforce backward compatibility during rollout, and continuously validate resilience assumptions (chaos testing) rather than assuming they hold.

Section C — REST APIs

Q141. What is REST and what are its architectural constraints? REST (Representational State Transfer) is an architectural style for distributed hypermedia systems defined by Roy Fielding, governed by six constraints: client-server separation, statelessness, cacheability, a uniform interface, a layered system, and (optionally) code-on-demand. An API satisfying these constraints is described as "RESTful."

Q142. REST vs SOAP. SOAP is a protocol with a strict XML envelope, built-in standards (WS-Security, WS-AtomicTransaction), and typically requires a WSDL contract — heavier but offering formal contracts and enterprise-grade features. REST is an architectural style, protocol-agnostic in principle but almost always over HTTP, lighter-weight, supports multiple formats (JSON commonly), and is generally simpler and more widely adopted for web/mobile APIs today.

Q143. HTTP methods: safety and idempotency. GET, HEAD, OPTIONS are safe (no side effects) and idempotent. PUT and DELETE are idempotent (repeating the same request yields the same end state) but not safe. POST is neither safe nor idempotent by default (each call can create a new resource). PATCH is generally not guaranteed idempotent unless explicitly designed to be.

Q144. PUT vs PATCH. PUT replaces the entire resource representation with the request payload (the client sends the full resource state). PATCH applies a partial update, modifying only the specified fields — typically smaller payloads, but requires a clear partial-update semantics (e.g., JSON Merge Patch or JSON Patch) to avoid ambiguity about how to interpret the diff.

Q145. HTTP status code categories. 1xx Informational, 2xx Success (200 OK, 201 Created, 202 Accepted, 204 No Content), 3xx Redirection (301, 304 Not Modified), 4xx Client Error (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests), 5xx Server Error (500, 502, 503, 504). Choosing precise codes is essential for clients to programmatically handle responses correctly.

Q146. Statelessness in REST. Every request from client to server must contain all information needed to understand and process it — the server holds no client session state between requests. This improves scalability (any server instance can handle any request, enabling easy horizontal scaling/load balancing) and reliability, at the cost of slightly larger requests (auth tokens, context repeated each call).

Q147. What is HATEOAS? Hypermedia As The Engine Of Application State — the constraint that a REST response should include links describing available next actions/related resources (e.g., a `cancel` link on an order only if it's cancellable), so clients navigate the API dynamically via returned links rather than hardcoding URL structures, similar to how a browser navigates a website via links.

Q148. Richardson Maturity Model. Describes REST API maturity in four levels: Level 0 (single URI, single HTTP method — essentially RPC-over-HTTP), Level 1 (multiple resource URIs, still one HTTP method typically POST), Level 2 (proper use of HTTP verbs and status codes per resource — where most "REST" APIs actually sit), Level 3 (full HATEOAS — hypermedia controls included).

Q149. Content negotiation in REST. Allows the same resource URI to be represented in multiple formats; the client specifies its preference via the `Accept` header (e.g., `application/json` vs `application/xml`), and the server responds with `Content-Type` indicating the format actually returned, returning `406 Not Acceptable` if it cannot satisfy the request.

Q150. API versioning strategies for REST. URI versioning (`/api/v1/orders` — simple, visible, but "pollutes" the URI), header versioning (custom header or `Accept: application/vnd.api.v2+json` — keeps URIs clean but less discoverable), and query-parameter versioning (`?version=2` — simple but easy to omit accidentally). URI versioning is the most common in practice for its simplicity and cacheability.

Q151. Why does idempotency matter for REST APIs? Networks are unreliable — a client may time out waiting for a response and retry a request that actually succeeded server-side. If the operation is idempotent (or protected by an idempotency key for POST), the retry is safe and won't create a duplicate resource or double-charge a payment, which is critical for reliability in distributed systems.

Q152. Pagination strategies. Offset-based (`?page=2&size=20` — simple but can skip/duplicate records if data changes between requests, and gets slow on large offsets), cursor/keyset-based (`?after=<opaque_cursor>` — stable and efficient even for huge datasets, doesn't support random page jumps), and

link-header-based (returning `next`/`prev` URLs directly, as GitHub's API does) which pairs well with HATEOAS.

Q153. Implementing API rate limiting. Choose an algorithm (token bucket is common — a bucket refills at a fixed rate and each request consumes a token, allowing bursts up to the bucket size while enforcing a steady average rate), implement it centrally at the gateway (backed by Redis for a distributed counter across instances), and return `429` with `Retry-After`/`X-RateLimit-*` headers when exceeded.

Q154. Authentication vs Authorization. Authentication answers "who are you?" (verifying identity — login, token validation). Authorization answers "what are you allowed to do?" (permission/role checks after identity is established). A request can be authenticated (valid token) yet still be denied via authorization (`403 Forbidden`) if the identified user lacks the required permission.

Q155. OAuth2 grant types. Authorization Code (for server-side/mobile apps with a user present — the most secure interactive flow, especially with PKCE), Client Credentials (machine-to-machine, no user context), Refresh Token (obtaining a new access token without re-authenticating the user), and the now-discouraged Implicit and Resource Owner Password Credentials grants (largely deprecated in modern OAuth2 security best practices).

Q156. JWT structure and usage. A JWT has three Base64URL-encoded, dot-separated parts: header (algorithm/type), payload (claims — subject, expiry, roles/custom claims), and signature (verifying integrity, computed over header+payload with a secret/private key). Because it's self-contained and cryptographically verifiable, a resource server can validate a JWT locally without a round-trip to the auth server, enabling stateless authentication.

Q157. API documentation with OpenAPI/Swagger. OpenAPI is a standard, machine-readable specification format describing an API's endpoints, request/response schemas, auth requirements, and examples. Swagger UI/Springdoc renders this spec as interactive, browsable documentation, and the spec can also drive client SDK generation, contract testing, and mock server generation — keeping documentation synchronized with the actual API via annotations or code-first generation.

Q158. HATEOAS vs plain REST. Plain (Level 2) REST relies on out-of-band documentation for clients to know which URLs/actions exist and when. HATEOAS embeds those affordances directly in responses as hypermedia links, in principle letting clients adapt to API changes dynamically — though in practice, full HATEOAS is uncommon because most clients are built against fixed documentation/SDKs anyway, and the added complexity often isn't worth the benefit for typical internal/B2B APIs.

Q159. Best practices for REST URI/resource naming. Use plural nouns for collections (`/orders`, not `/order` or `/getOrders`), represent hierarchy via nesting (`/customers/{id}/orders`), avoid verbs in URIs (the HTTP method already conveys the action), use lowercase-hyphenated paths, and keep URIs resource-oriented rather than mirroring RPC-style function calls.

Q160. Error handling best practices / RFC 7807. Return a consistent, machine-parsable error body across all endpoints, ideally following RFC 7807 "Problem Details for HTTP APIs" (`type`, `title`, `status`, `detail`, `instance` fields), paired with the correct HTTP status code — never return `200 OK` with an error message buried in the body, as that breaks standard HTTP client error handling.

Q161. What is CORS and why is it needed? Browsers enforce the Same-Origin Policy, blocking JavaScript from making requests to a different origin than the page was served from, unless the target server

explicitly opts in via CORS response headers (`Access-Control-Allow-Origin`, etc.). This is essential for browser-based SPAs calling an API hosted on a different domain/port than the frontend.

Q162. Securing REST APIs. Always use HTTPS/TLS to encrypt data in transit, authenticate requests (OAuth2/JWT for user-facing, API keys or mTLS for service-to-service), authorize per-endpoint/resource, validate and sanitize all input (preventing injection attacks), apply rate limiting, and avoid leaking sensitive details in error messages/stack traces.

Q163. API Gateway's role for REST APIs specifically. Terminates client TLS connections, authenticates/authorizes requests before they reach backend services, routes to the correct backend, enforces rate limits/quotas, can aggregate multiple backend calls into one client-facing response, and provides a single place for centralized API logging/analytics/versioning policy.

Q164. Sync vs async REST design (webhooks, polling). For operations that take longer than a typical request timeout, a synchronous REST call isn't appropriate. Alternatives: polling (client repeatedly calls a status endpoint after an initial `202 Accepted`), or webhooks (the server calls back to a client-registered URL once processing completes) — webhooks are more efficient (push vs. poll) but require the client to expose a reachable endpoint.

Q165. HATEOAS link relations in Spring HATEOAS. Spring HATEOAS provides `WebMvcLinkBuilder.linkTo(...).withRel("relName")` to construct typed links (e.g., "self", "cancel", "next") attached to an `EntityModel`/`CollectionModel`, which get serialized (often per the HAL media type, `application/hal+json`) with an `_links` section that clients can programmatically follow.

Q166. REST API caching (ETag, Cache-Control). `Cache-Control` headers (`max-age`, `no-cache`, `private`/`public`) tell clients/intermediary caches how long a response may be reused. `ETag` is a hash/version identifier of the resource; a client can send `If-None-Match: <etag>` on subsequent requests, and the server responds `304 Not Modified` (no body) if unchanged, saving bandwidth while still validating freshness.

Q167. Idempotency key pattern for POST. Since POST is not inherently idempotent, clients generate a unique idempotency key (often a UUID) and send it in a header on a create request; the server stores the key with the result of the first successful processing and, if the same key arrives again (e.g., due to a client retry), returns the cached original response instead of creating a duplicate resource.

Q168. REST vs GraphQL. REST exposes multiple fixed endpoints, each returning a predetermined shape — simple, cacheable via HTTP semantics, but can lead to over-fetching (extra unused fields) or under-fetching (needing multiple round trips). GraphQL exposes a single endpoint where clients specify exactly the fields/nested relations they need in one query, reducing over/under-fetching, at the cost of more complex server-side query resolution, caching, and rate-limiting.

Q169. Throttling vs quota. Throttling limits request *rate* over a short window (e.g., 100 requests/minute) to smooth bursts and protect infrastructure. Quota limits total *volume* over a longer billing/usage period (e.g., 1 million requests/month), typically tied to a pricing tier — both can apply simultaneously to the same API consumer.

Q170. Handling bulk operations in REST APIs. Provide a dedicated batch endpoint (e.g., `POST /orders/batch`) accepting an array of items in one request, returning a per-item status array (since some

items may succeed and others fail) rather than forcing N separate round-trips — improving throughput and reducing network overhead for bulk client use cases.

Q171. REST API testing (Postman, RestAssured). Postman provides a GUI for manually building/running requests, collections, and automated test scripts (assertions on status/body), plus a CLI runner (Newman) for CI. RestAssured is a Java DSL for writing fluent, readable HTTP assertions directly in automated test suites (`given().when().then()`), integrating naturally into JUnit-based test pipelines alongside the rest of a Java codebase.

Q172. Contract-first vs code-first API design. Contract-first defines the OpenAPI spec before writing any implementation code, letting consumer teams start building against a mock/stub immediately and generating server stubs/client SDKs from the spec — enforces API design discipline upfront. Code-first writes the implementation first and generates the OpenAPI spec from annotations afterward — faster to start but risks the API design being an accidental byproduct of implementation rather than a deliberate contract.

Q173. API mocking. Simulating an API's responses (via tools like WireMock, Postman Mock Server, or a Prism server generated from an OpenAPI spec) before or independent of the real implementation, letting frontend/consumer teams develop and test in parallel with backend development, and enabling isolated tests that don't depend on a real, possibly-flaky, downstream service.

Q174. Richardson Maturity Model Level 3 (HATEOAS) in more depth. At Level 3, responses don't just use proper resources/verbs (Level 2) but also embed hypermedia controls describing valid next actions given the resource's current state — e.g., an order response includes a `pay` link only while unpaid, and a `cancel` link only while cancellable — letting the API itself communicate the state machine rather than requiring client-side hardcoded business rules.

Q175. 401 vs 403. `401 Unauthorized` means the request lacks valid authentication credentials (the client isn't proven to be who it claims — should typically include a `WWW-Authenticate` header). `403 Forbidden` means the client's identity is known/valid, but it doesn't have permission to perform the requested action on that resource.

Q176. Designing a search/filter API endpoint. Use query parameters for filtering (`?status=active&category=books`), sorting (`?sort=createdAt,desc`), and pagination (`?page=0&size=20`), keeping the base resource path clean (`GET /products`). For very complex search criteria, consider a dedicated `POST /products/search` endpoint accepting a structured query body instead of an unwieldy long query string.

Q177. ETags for optimistic concurrency control. A client fetches a resource along with its current `ETag`; when updating, it sends that ETag in an `If-Match` header. If another client modified the resource in the meantime (ETag mismatch), the server rejects the update with `412 Precondition Failed` instead of silently overwriting the other change — a lightweight, standard alternative to server-side version-number fields for detecting lost updates.

Q178. Webhooks and their use cases. Instead of a client repeatedly polling for status, the server (or a third-party service like a payment provider) sends an HTTP POST to a client-registered callback URL when a relevant event occurs (payment succeeded, file processed). This is far more efficient than polling, but requires the receiving client to expose a publicly reachable, authenticated endpoint and handle retries/duplicate deliveries idempotently.

Q180. REST vs RPC-style APIs. REST models the API around resources (nouns) manipulated via standard HTTP verbs, aiming for a uniform, self-descriptive interface. RPC-style APIs (including gRPC, or REST APIs that are "RESTful in name only") expose action-oriented endpoints (`/createOrder`, `/cancelOrder`) closer to remote method calls — simpler to map directly to business operations that don't fit neatly into CRUD-on-a-resource semantics, at the cost of losing some of REST's uniform-interface benefits (caching, standard tooling).

Q181. What is a DTO and why not expose entities directly? A Data Transfer Object is a plain object shaped specifically for the API contract, decoupled from the internal JPA entity. Exposing entities directly risks leaking internal DB structure/lazy-loading issues into the API contract, tightly coupling your persistence schema to your public API (making either hard to evolve independently), and can accidentally expose sensitive fields not meant for clients.

Q182. Input validation best practices for REST APIs. Validate at the API boundary before any business logic executes (using Bean Validation annotations on request DTOs), reject invalid requests early with `400 Bad Request` and a clear, field-level error message, whitelist rather than blacklist acceptable input where possible, and never rely solely on client-side validation since API clients can bypass it entirely.

Q183. API Gateway request/response transformation. Gateways can rewrite paths/headers, aggregate multiple backend responses into one client response, translate between protocols (e.g., expose REST externally while calling gRPC internally), and strip/add fields — decoupling the externally-published API contract from internal service implementation details, so backend services can evolve without breaking external consumers.

Q184. REST API response envelope design. Modern practice generally favors returning the resource representation directly as the top-level JSON body (relying on HTTP status codes for success/error signaling) rather than wrapping every response in a generic envelope (`{success: true, data: {...}}`), since that envelope duplicates what status codes already convey and complicates client deserialization — exceptions include APIs needing consistent pagination metadata alongside the data array.

Q185. Media types (MIME types) in REST. A media type (e.g., `application/json`, `application/xml`, `application/hal+json`) tells the client/server how to interpret the body's format/structure. It's communicated via the `Content-Type` header (what's being sent) and negotiated via the `Accept` header (what the client can understand in the response), forming the basis of REST's content-negotiation constraint.

Q186. JSON Patch (RFC 6902) vs JSON Merge Patch (RFC 7396). JSON Patch describes an explicit sequence of operations (`add`, `remove`, `replace`, `move`, `test`) to transform a document — precise but more verbose. JSON Merge Patch is a simpler diff-style document merged into the target (any field present overwrites, `null` removes a field) — easier to construct but can't express operations like array element removal by position unambiguously.

Q187. API monitoring and SLA/SLO. Monitoring tracks real-time metrics (latency percentiles, error rate, throughput) against defined Service Level Objectives (e.g., "99.9% of requests succeed in under 200ms"), which roll up into a Service Level Agreement (a contractual commitment to consumers). Alerting should be based on SLO burn-rate, not just raw thresholds, to catch degrading trends before a full outage.

Q188. Long-running operations pattern (202 Accepted + polling). For operations too slow for a synchronous response, the server immediately returns `202 Accepted` with a `Location` header pointing to a status resource; the client polls that resource (or is notified via webhook) until it reports completion, at which point the response includes the final result or a link to it — avoiding tying up client/server connections for the full operation duration.

Q189. REST API vs "Web Service" generally. "Web Service" is a broader umbrella term for any service exposed over a network via standard web protocols, historically dominated by SOAP/WSDL-based services. REST is one specific, now-dominant architectural style for building web services (alongside SOAP, GraphQL, gRPC) — so "REST API" is a type of web service, not a synonym for the entire category.

Q190. Security headers relevant to REST APIs. `Strict-Transport-Security` (HSTS, forces HTTPS), `X-Content-Type-Options: nosniff` (prevents MIME-sniffing attacks), `Content-Security-Policy` (mainly for browser-rendered content but relevant if the API serves any HTML/docs), and appropriate `Cache-Control` on sensitive endpoints (`no-store` to prevent caching of tokens/PII) — collectively hardening the API against common web-layer attacks.

Q191. API key vs OAuth2 token — when to use which. API keys are simple, static, long-lived credentials suited for coarse-grained service identification (e.g., a third-party integration or internal service-to-service call) but offer weaker security (no built-in expiry/scoping) and are harder to rotate safely. OAuth2 tokens are short-lived, scoped, and support delegated user-context authorization — the appropriate choice whenever an API acts on behalf of an end user or needs fine-grained, revocable access control.

Q192. Designing idempotent DELETE operations. `DELETE /resources/{id}` is idempotent by definition — the first call removes the resource and returns `204 No Content` or `200 OK`; subsequent calls to the same URL should return `404 Not Found` (since the resource no longer exists) rather than an error, because the end state ("resource does not exist") is identical regardless of how many times the delete was invoked.

Q193. Significance of the `Location` header in `201 Created`. When a POST successfully creates a new resource, the response should be `201 Created` along with a `Location` header containing the URI of the newly created resource, so the client immediately knows where to `GET`/`PUT`/`DELETE` that specific resource going forward without needing to construct the URL itself.

Q194. Versioning breaking changes without breaking existing clients. Introduce the breaking change in a new API version while keeping the old version fully operational in parallel, use additive-only changes to the existing version wherever possible (new optional fields don't break well-behaved clients that ignore unknown fields), communicate the migration path and timeline clearly, and only retire the old version once consumer migration is confirmed or the sunset date is reached.

Q195. API Gateway aggregation pattern. The gateway (or a dedicated BFF) receives one client request, fans it out to multiple backend microservices in parallel, and combines their responses into a single unified payload before returning it to the client — reducing the number of round trips a client (especially a mobile client on a slow network) needs to make to assemble a complete view.

Q197. Significance of `Accept` and `Content-Type` headers. `Content-Type` describes the media type of the data actually being sent in the current request/response body. `Accept` (sent by the client) specifies which media type(s) the client is willing to receive in the response. Mismatches lead to `415 Unsupported Media Type` (bad `Content-Type`) or `406 Not Acceptable` (server can't satisfy the requested `Accept`).

Q198. Rate-limit response headers. Common (though not universally standardized) headers: `X-RateLimit-Limit` (max requests allowed in the window), `X-RateLimit-Remaining` (requests left in the current window), `X-RateLimit-Reset` (when the window resets), and `Retry-After` (seconds to wait, typically sent with a `429` response) — letting well-behaved clients self-throttle proactively instead of blindly hitting the limit.

Q199. API sandboxing for third-party developers. A sandbox is an isolated environment mirroring production API behavior (using test data/fake payment processing, etc.) that third-party developers can integrate against safely during development, without risk of affecting real data or incurring real-world side effects (charges, emails) — essential for platforms with external developer ecosystems (payment gateways, social platforms).

Q200. Best-practices checklist for production-grade REST APIs. Use proper HTTP verbs/status codes consistently; version deliberately and deprecate gracefully; secure with HTTPS + OAuth2/JWT; validate all input; return consistent, structured error responses (RFC 7807); support pagination/filtering/sorting on collections; document with OpenAPI and keep it in sync; enforce rate limiting; design idempotent write operations with idempotency keys where relevant; support caching (ETag/Cache-Control) where appropriate; monitor with metrics/logs/traces against defined SLOs; and maintain backward compatibility as the default posture for change.

End of document — 200 questions across Spring Boot (70), Microservices (70), and REST APIs (60).